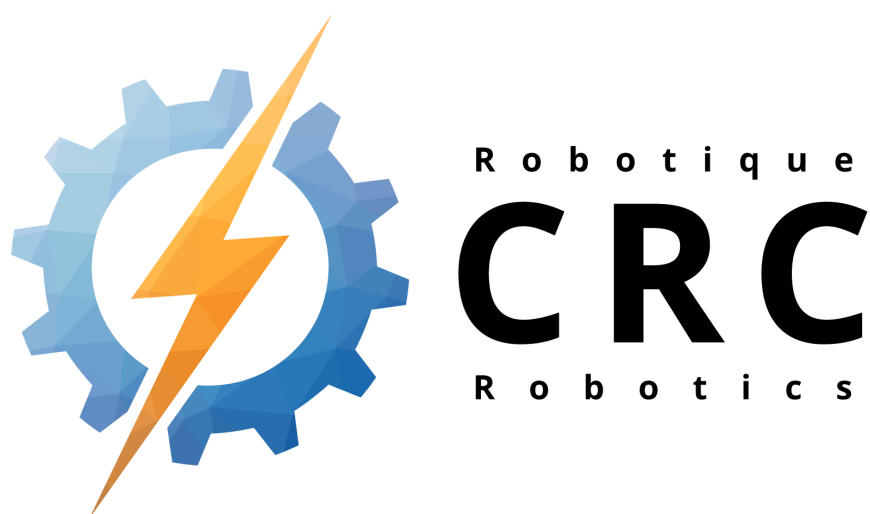




CRC
**COMPÉTITION DE
PROGRAMMATION**



**LIVRET DE PROBLÈMES
BLOC B**

QUELQUES NOTES

- Les règles complètes sont dans la section 4 du livret des règlements. **N'oubliez pas les multiplicateurs!**
- Vous avez jusqu'à **19h59** pour remettre vos solutions. Remettez la structure de fichier comme vous l'avez reçu avec vos solutions dans les fichiers .py associés.
- N'hésitez pas à nous poser des questions et à communiquer avec les membres de votre équipe de programmation. Ce n'est pas un examen!
- **On vous donne des templates faciles à utiliser pour votre code. Vous devez les utiliser comme des templates similaires sont utilisés pour la correction!**

UTILISATION DU FICHIER MODÈLE

- Tous les fichiers possèdent une fonction `solve()` dans lequel vous devez résoudre le problème. **NE CHANGEZ PAS LE NOM DE CETTE FONCTION!**
- Pour rouler vos tests, dans un terminal (nous vous conseillons l'utilisation de vscode) écrivez la ligne de commande: `pytest` pour rouler les tests. Si vous faites la commande `pytest` dans le répertoire contenant tous les fichiers, tous les tests seront exécutés.
- Pour rouler les tests d'un problème spécifique, naviguez dans le répertoire de la question et exécutez la commande `pytest` pour rouler les tests du problème. Vous pouvez faire la même chose pour une section entière!

STRUCTURE

Chaque problème contient une petite mise en situation comme celle-ci expliquant les fondements du problème et donnant les bases nécessaires pour résoudre celui-ci. Chaque problème contient aussi la répartition des points pour le problème.

Spécifications d'entrée et de sortie:

Dans cette section, on retrouve les caractéristiques des entrées qui peuvent être fournies au code en question ainsi que les critères attendus pour les sorties du programme.

Exemple d'entrée et de sortie:

Dans cette section se trouve un exemple d'entrée (parfois constitué lui-même de plusieurs sous-exemples) pour que vous puissiez tester votre code. L'exemple de sortie donne donc la réponse attendue pour cette entrée.

Explication de la première sortie:

Si le problème n'est toujours pas clair après la mise en situation, l'explication de la première sortie sert parfois à démêler le tout en expliquant comment la première entrée est traitée et en montrant le chemin menant à cette réponse.

Table des matières

Trains et Métros (TM)	4
TM1: Tramway: (30 points)	4
TM2: Trajet de métro! (15 points)	6
TM3: Loop Line (25 points)	7
TM4: Qui n'a aucun billet?? (111 points)	9
Mythologies (MY)	11
MY1: Charybdis (20 points)	11
MY2: Thor (35 points)	13
MY3: Le fil d'Ariane (70 points)	16
MY4: Retour à Ithaca (90 points)	19
Olympique d'hiver (WO)	22
WO1: Patinage de vitesse (25 points)	22
WO2: Bobsleigh (45 points)	24
WO3: Saut à ski (15 points)	26
WO4: Il faut être le meilleur pour une médaille? Right? (30 points)	28
WO5: Parfaite ligne d'attaquants (55 points)	31

Trains et Métros (TM)

TM1: Tramway: (30 points)

Lors d'un voyage à l'étranger, vous devez utiliser le système de tramway local pour vous déplacer. Vous aurez comme entrée une liste de *string* représentant le réseau de tram que vous pouvez utiliser sous forme d'arbre. Vous devrez ensuite retourner une *string* contenant les mouvements à faire pour naviguer l'arbre jusqu'à votre destination. Les arbres peuvent être de grandeurs variables d'un exemple à l'autre, mais la rangée de destinations possibles est toujours remplie. (voir ci-dessous pour des exemples d'arbres)

Pour chaque embranchement que vous croisez, vous devrez mettre dans la *string* de sortie "L" (pour *Left* ou gauche) ou "R" (pour *Right* ou droite) selon la direction que vous prenez en descendant l'arbre. **Les directions gauche/droite sont données par rapport à vous qui visionnez cet arbre: nous n'utilisons pas les directions selon le point de vue du tramway.**

Spécifications d'entrée:

En entrée vous recevrez:

- **network:** une *liste* contenant des *strings* représentant les lignes de tram,
- **destination:** un *int* qui est la destination à atteindre.

Spécifications de sortie:

En sortie vous devrez fournir:

- une *string* avec les directions ("R", "L") à prendre pour se rendre au target.

Exemple d'entrée:

```
network= [ "      /\      ",
           "    /  \    ",
           "  /      \  ",
           " /\      /\ ",
           "6  3  4  7" ]
destination=3
```

```

network= [ "
           /\
          /\
         /\
        /\
       /\
      /\
     /\
    /\
   /\
  /\
 /\
/\
"6  3  4  7  9  0  2  1"]

destination=0

```

```

network= [ "
           /\
          /\
         /\
        /\
       /\
      /\
     /\
    /\
   /\
  /\
 /\
/\
"6  8  5  2  3  0  1  7"]

destination=1

```

Exemple de sortie:

```

"LR"
"RLR"
"RRL"

```

Explication de la première sortie:

Voici l'arbre de la première sortie:

```

[ "
   /\
  /\
 /\
 /\
"6  3  4  7"]

```

Nous devons atteindre la destination 3 et il faudra traverser 2 embranchements pour se rendre.
 Nous devons donc descendre la partie gauche de l'arbre en haut, puis la partie droite => "LR".

TM2: Trajet de métro! (15 points)

Les écoles doivent se rendre à la compétition pour participer! Vous devez déterminer combien de temps l'école prend pour se rendre à la compétition!

Un métro passe à chaque station aux 5 minutes à partir du moment où vous commencez votre trajet et le train prend 2.5 minutes pour faire une seule station.

Spécifications d'entrée:

En entrée vous recevrez:

- **stations:** un *array* de *int* du nombre de stations à faire pour chaque ligne, la station de départ est incluse pour chaque ligne à faire

Spécifications de sortie:

En sortie vous devrez fournir:

- un *float* du nombre de minutes que l'équipe prend pour se rendre à la compétition.

Exemple d'entrée:

[8, 17]

[27]

[10, 10]

Exemple de sortie:

60.0

65.0

47.5

Explication de la première sortie:

Dans la première ligne, il y a 7 stations à faire puisque la station de départ ne compte pas. ce qui prend 17.5 minutes. Il faut attendre 2.5 minutes pour attraper le prochain métro lors du transfert, comme il passe à tous les multiples de 5 minutes.. Ensuite, sur la seconde ligne, il y a 16 stations à faire et cela prend 40 minutes pour un total de 60 minutes.

TM3: Loop Line (25 points)

Vous êtes responsable de concevoir le système de métro d'une ville. Vous avez d'abord choisi de construire toutes les stations, puis de les relier avec des lignes appropriées. En faisant cela, vous vous êtes rendu compte que vous pouvez connecter N stations du réseau en une boucle: le train partira de la station 1, puis ira à la station 2, à la station 3, ..., jusqu'à atteindre la station N , puis reviendra à la station 1.

Lors d'un test de la voie, vous vous êtes ennuyé et avez décidé d'essayer de deviner si votre train circule dans le sens horaire ou antihoraire sur la boucle. Vous avez décidé de vous lancer ce défi sans regarder la carte. Heureusement, vous avez une boussole sur votre téléphone et vous avez conçu un système de signalisation tous les kilomètres.

Après le départ du train de la station 1, chaque fois que vous voyez un signal, vous consultez la boussole et notez la direction indiquée. Vous répétez ce processus jusqu'à votre retour à la station 1. Vous obtenez alors une liste des directions relevées et vous souhaitez déterminer si le train circule dans le sens horaire ou antihoraire.

Spécifications d'entrée:

En entrée vous recevrez:

- **directions**: une *string*, qui est une séquence de N(Nord), E(Est), S(Sud) et W(Ouest), qui représentent la direction de la ligne

Spécifications de sortie:

En sortie vous devrez fournir:

- une seule *string*, qui est "CW" (si le train va en sens horaire) ou "CCW" (si le train va en sens antihoraire).

Exemple d'entrée:

NNEEESSWW

SSSEENWNENWW

WSWNWWNNESEEEENWWNNESENNEEESWWSEESWWSEESWWSESWWNN

Exemple de sortie:

CW

CCW

CW

Explication de la première sortie:

Nous avons @ qui indique la station initiale et * qui représente le chemin et les flèches sont l'ordre de parcours. Ceci nous donne la visualisation suivante:

```
*>*>*>*  
^      v  
*      *  
^      v  
*      *  
@<*<*<*
```

Dans ce cas, il est très simple de voir que nous allons en sens horaire!

TM4: Qui n'a aucun billet?? (111 points)

Vous êtes un contrôleur du métro de Montréal et vous voulez savoir à quelle station est associée "Aucun" billet de transport. Vous avez quelques informations sur les stéréotypes des différentes stations de métro et vous devrez trouver laquelle est associée avec le titre de transport le plus commun étant "Aucun". Prenez les informations que vous avez et organisez-les dans un tableau pour retrouver toutes les informations sur les stations. Dans notre exemple, les stations sont considérées les 5 comme voisines de gauche (station0) à droite (station4).

Vous devrez remplir une matrice 2D contenant les informations suivantes: la position de 0 à 4, la couleur de la station, le nom de la station, le sac le plus commun, la boisson la plus commune, mais surtout le titre de transport le plus acheté! Avec une série d'indices, vous devrez reconstruire la matrice 2D et nous la redonner.

Les entrées ne sont pas dans le bon ordre, elles sont aléatoires!

Ligne: Bleu, Vert, Orange, Jaune, Rose

Station: Snowdon, Berri-UQAM, Vendôme, Licorne, Atwater

Sac: Sac à dos, Tote bag, Sac à main, Bandoulière, Sac plastique

Boisson: Tea, Jus, Coffee, Eau, Lait

Titre: Weekend illimité, Soirée illimité, Aucun, 1 passage, Mensuel

Spécifications d'entrée:

En entrée, vous recevrez une série d'instructions en string sous une des formes suivante:

- élément 1 = élément 2 (les deux éléments sont dans la même colonne)
- élément 1 -> élément 2 (l'élément 1 est une colonne à gauche de l'élément 2)

Spécifications de sortie:

En sortie, vous devez donner une matrice 2D remplie de strings. Vous mettrez en premier Station0 jusqu'à Station4, suivi de la couleur de la ligne, le nom de la station, la nourriture associée, la boisson associée et le titre de transport associé. Ceci est vrai pour chaque station partant de la station à la position 0 jusqu'à la station à la position 4.

ex:[["Station0", Ligne0, Nom0, Sac0, Boisson0, Titre0], ["Station1", Ligne1, ...], ...]

L'entrée:

Vendôme = Orange

Licorne = Soirée illimité

Snowdon = Sac à dos

Vert -> Rose

Vert = Coffee

Sac à main = Mensuel

Jaune = Bandoulière

Station2 = Lait
Station0 = Berri-UQAM
Weekend-illimité -> Sac à dos
Bandoulière -> 1 passage
Sac plastique = Jus
Atwater = Tote bag
Berri-UQAM -> Bleu
Eau -> Sac à dos

La sortie:

Un array de 5 éléments (stations) avec chacun des 6 éléments ("StationX", Ligne, Nom station, Sac, Boisson, Titre transport) C'est à vous de nous la donner! (Oui il y a une seule réponse valide)

	Station0	Station1	Station2	Station3	Station4
Ligne					
Nom					
Sac					
Boisson					
Titre					

Explication de la sortie:

Pour plus d'explications allez voir Frank.

Mythologies (MY)

MY1: Charybdis (20 points)

Charybdis est un monstre présent dans l'Odyssée. Elle crée un tourbillon d'eau et il faut résister à ce tourbillon ou c'est la fin. Trouvez à partir d'où il est sécuritaire de naviguer.

Elle va être représentée par la forme d'un beigne, le rayon du grand cercle va toujours être 1 (le centre est en (0,0) en tout temps), le rayon du tourbillon (et donc du petit cercle) va changer pour chaque entrée. À 0.15 de distance du cercle intérieur, vous êtes dans le tourbillon et serez menés à votre perte. Vous devez déterminer combien de positions données sont en sécurité.

Note : Les positions peuvent être hors du cercle extérieur

Formule pour le cercle:

$$x^2 + y^2 = r^2$$

Spécifications d'entrée:

Vous allez recevoir :

- ***whirlpool***: un *float* du rayon du tourbillon
- ***positions***: une *liste* de *tuples* de *float* pour les positions

Spécifications de sortie:

En sortie vous devez retourner un *int* du nombre de positions sécuritaires.

Exemple d'entrée:

```
whirlpool = 0.45
```

```
positions = [(-0.3, 0.5), (0.25, -0.4), (1, -0.75)]
```

```
whirlpool = 0.24
```

```
positions = [(-0.1, -0.05), (0.3, -0.2), (0.0, 0.39), (0.1, -0.23)]
```

```
whirlpool = 0.84
```

```
positions = [(-0.45, 0.45), (0.31, -0.15), (0.5, -0.5), (-0.71, -0.7), (75.5, 3.45), (0.99, 0.01)]
```

Exemple de sortie:

1
0
3

Explication de la première sortie:

Dans le premier exemple, le tourbillon a un rayon de 0.45, donc tout ce qui est dans un rayon de 0.6 va dans le tourbillon. La position $(-0.3; 0.5)$ a un rayon d'environ 0.58 ce qui signifie que cette position n'est pas sécuritaire. La position $(0.25; -0.4)$ a un rayon d'environ 0.47 donc c'est aussi non sécuritaire. La 3^e position $(1; -0.75)$ a un rayon de 1.25 et donc est sécuritaire! Donc une seule position est sécuritaire.

MY2: Thor (35 points)

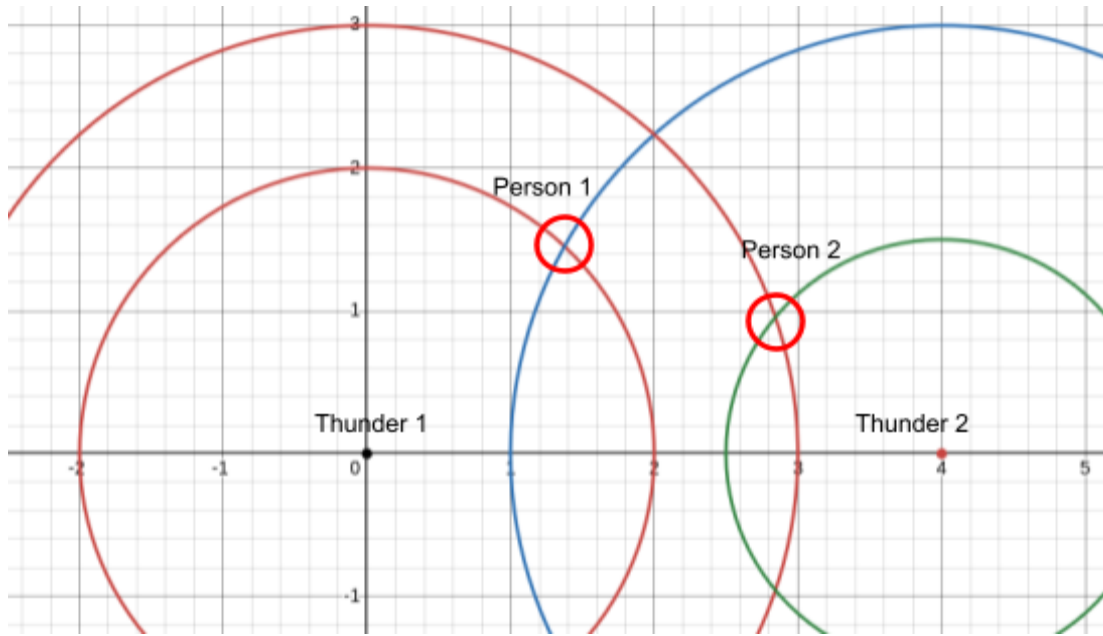
Dans la mythologie nordique, Thor est le Dieu de la foudre et des éclairs. Son marteau mjolnir est reconnu comme la cause de la foudre. Les éclairs seraient donc causés par Thor.

Quand des éclairs se produisent, vous avez sûrement déjà remarqué qu'on voit l'éclair bien avant d'entendre le son. Si on connaît la différence de temps entre le moment où l'on voit l'éclair et celui où on l'entend, on peut déterminer la distance à laquelle l'éclair a eu lieu!

Nous avons deux éléments à prendre en compte dans ce problème, le temps nécessaire pour que la lumière de l'éclair se rend jusqu'à nous et le temps que le son de l'éclair prend pour se rendre jusqu'à nous. Tout d'abord, pour la lumière, elle se déplace à 300 000 km/s alors que le son se déplace à environ 340 m/s dans l'air. Comme la lumière se déplace extrêmement vite, on peut considérer que le temps requis pour qu'elle se rende jusqu'à nous est instantané. On peut donc, avec le temps entre la lumière et le son déterminer notre distance avec l'éclair en divisant ce temps par la vitesse du son pour obtenir notre distance.

Si nous avons 2 personnes qui entendent et voient les 2 mêmes éclairs et savent précisément le temps entre la lumière de l'éclair et le son de l'éclair, il est possible de trianguler leur position pour savoir exactement la distance entre les deux personnes.

Bref, si nous avons un éclair à un endroit et un second exactement 4 km plus loin, si les deux personnes peuvent déterminer leur distance de l'éclair, elles peuvent déterminer leur distance entre elles. Une dernière hypothèse doit être ajoutée pour avoir une réponse unique. Comme être à une distance d'un point permet d'être à deux endroits possibles pour chacune des deux personnes, il faut choisir l'option où les deux sont du même bord du centre et sont donc le plus près possible l'un de l'autre.



Spécifications d'entrée:

En entrée vous recevrez:

- ***thunder_1***: un *tuple* de *float* du temps en secondes entre la lumière et le son du tonnerre pour les deux personnes par rapport au premier éclair (position 0,0)
- ***thunder_2***: un *tuple* de *float* du temps en secondes entre la lumière et le son du tonnerre pour les deux personnes par rapport au deuxième éclair (position 4,0), qui se trouve à 4 km de distance du tonnerre initial

Spécifications de sortie:

En sortie, vous devrez donner la distance en mètres entre les deux personnes avec une précision de 2 décimales après la virgule.

Exemple d'entrée:

`thunder_1 = (5.88, 8.82)`

`thunder_2 = (8.82, 4.41)`

`thunder_1 = (2000/340, 2100/340)`

`thunder_2 = (2500/340, 2400/340)`

`thunder_1 = (1000/340, 2000/340)`

`thunder_2 = (3000/340, 2000/340)`

Exemple de sortie:

1549.45

112.62

1000.00

Explication de la première sortie:

Dans le premier exemple, nous avons la personne 1, qui entend le premier éclair 5,88 secondes après l'avoir vu, et la personne 2, qui entend l'éclair 8.82 secondes après l'avoir vu. De ceci, comme la vitesse du son est de 340 m/s dans ce problème, nous avons une distance de:

$$d_{1,1} = 340 * 5.88 = 1999.2 \text{ m}$$

$$d_{1,2} = 340 * 8.82 = 2998.8 \text{ m}$$

Nous pouvons faire pareil pour trouver la distance des deux personnes par rapport au 2e éclair.

$$d_{2,1} = 340 * 8.82 = 2998.8 \text{ m}$$

$$d_{2,2} = 340 * 4.41 = 1499.4 \text{ m}$$

On a maintenant la distance entre nos deux personnes et chacun des deux éclairs et nous avons dans le problème la distance entre les deux éclairs qui est d'exactement 4000m.

Avec quelques équations de cercles que je vous donne pour vous simplifier la vie... voici la position en x et y selon la distance de deux cercles.

$$x = (d_{e1}^2 - d_{e2}^2 + 4000^2) / (2 * 4000)$$

$$y = \sqrt{d_{e1}^2 - x^2}$$

Nous pouvons ainsi avoir la position en x et y de nos deux observateurs. Ceci nous permet de trouver leur distance avec la formule de distance en 2D:

$$distance = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

MY3: Le fil d'Ariane (70 points)

Dans la mythologie grecque, Ariane donne un fil à Thésée pour l'aider à sortir du labyrinthe après avoir vaincu le Minotaure. L'idée est simple: en entrant dans le labyrinthe, on déroule un fil pour pouvoir revenir sur ses pas et retrouver la sortie.

Vous entrez dans un labyrinthe par une unique entrée, vous devez atteindre un objectif, puis ressortir par la même entrée. Le labyrinthe contient plusieurs chemins possibles, des intersections et des culs-de-sac. Contrairement à un simple couloir, il est parfois impossible d'éviter de repasser par certaines cases.

Votre objectif est de déterminer le nombre minimum de cases qui devront obligatoirement être visitées deux fois soit une première fois en entrant dans le labyrinthe et en vous rendant à l'objectif et une seconde fois en revenant vers la sortie initiale. La case d'entrée E et la case d'objectif O ne comptent pas comme étant visitées 2 fois.

On suppose que :

- vous choisissez toujours un chemin optimal,
- vous connaissez parfaitement le labyrinthe,
- vous pouvez emprunter n'importe quel chemin libre,
- mais vous ne pouvez pas traverser les murs.

Le labyrinthe est représenté par un tableau de chaînes de caractères :

- # représente un mur
- . représente une case libre
- E représente l'entrée du labyrinthe (une seule)
- O représente l'objectif à atteindre (un seul)

L'entrée E est aussi la sortie : vous devez y revenir après avoir atteint O.

Les déplacements sont possibles uniquement dans les 4 directions cardinales :haut, bas, gauche, droite

Spécifications d'entrée:

En entrée vous recevrez:

- ***labyrinth***: un *array* de strings représentant le labyrinthe

Spécifications de sortie:

En sortie vous devrez fournir:

- un *int* représentant le nombre minimal de cases qui devront être visitées deux fois, peu importe l'ordre, pour aller de E à O puis revenir à E.

Exemple d'entrée:

```
[
  "#####",
  "#E...#",
  "#.#.#.#",
  "#.#...#",
  "#.###.#○.#",
  "#.....#",
  "#.#####.",
  "#####"
]
[
  "#####",
  "#E.....#",
  "#.###.#.#.###.#",
  "#...#.#.#...#.#",
  "###.#.#.###.#.#",
  "#...#.....○.#.#",
  "#.#####.#####.",
  "#####"
]
[
  "#####",
  "#E...#",
  "#.#.#.#○#",
  "#.#.....#",
  "#####"
]
```

Exemple de sortie:

8
4
6

Explication de la première sortie:

Voici un premier moyen de se rendre du début à la fin du labyrinthe:

```
[
"#####",
"#E..#...#",
"#.#.#.#.#",
"#.#...#.#",
"#.###.#O.#",
"#.....#..#",
"#.#####..#",
"#####"]
```

Voici le moyen de revenir avec le moins de cases qui sont utilisées 2 fois:

```
[
"#####",
"#E..#...#",
"#.#.#.#.#",
"#.#.#.#.#",
"#.###.#O.#",
"#.....#..#",
"#.#####..#",
"#####"]
```

Voici les 8 cases qui sont utilisées obligatoirement à l'aller et au retour

```
[
"#####",
"#E..#...#",
"#.#.#.#.#",
"#.#.#.#.#",
"#.###.#O.#",
"#.....#..#",
"#.#####..#",
"#####"]
```

MY4: Retour à Ithaca (90 points)

L'*Illiade* et l'*Odyssée* sont les deux grands récits du poète grec Homère et sont deux des récits les plus connus de la mythologie grecque. Ce que certains ne savent peut-être pas est que le deuxième est la suite directe du premier et nous raconte les 10 ans de périples qu'Ulysse, roi d'Ithaca, a subi pour revenir chez lui de la guerre de Troie. Le premier problème de cette catégorie porte justement sur un des périples qu'il a rencontrés, soit Charybde. Nous nous intéresserons plus au voyage marin d'Ulysse en tant que tel.

Pendant plusieurs parties de son voyage, des dieux mécontents de son arrogance font déferler tempêtes et vents puissants sur la flotte d'Ulysse et il s'échoue avec ses hommes sur plusieurs îles avant de finalement se rendre chez lui. **Compte tenu un archipel d'îles spécifiques (le même archipel pour tous les exemples), une vitesse initiale et une série de vents, vous devrez trouver la durée de fin du voyage d'Ulysse en heures.**

(Voir explication pour un dessin) Ulysse partira du point (0,0) en bas à gauche du plan cartésien et devra atteindre l'île d'Ithaca, qui est un carré délimité par les points (90,100), (100,100), (100,90) et (90,90). Le bateau ne sortira jamais de la région du plan cartésien décrite par $0 \leq x \leq 100$ et $0 \leq y \leq 100$. Les distances sont données en kilomètres et les vitesses, en km/h. Pour déterminer le mouvement du bateau après un temps "t" selon une vitesse initiale " v_0 ", une position initiale " x_0 " et un vent " w ", nous définissons la force qui agira sur le bateau:

$$\vec{F} = mk(\vec{w} - \vec{v})$$

Ne vous en faites pas: nous avons fait les calculs nécessaires pour obtenir les équations du mouvement à votre place! **Il vous faudra calculer les positions et vitesses en x indépendamment d'en y, ou vous éprouverez des difficultés.** La position du bateau après un temps "t" sous vent constant peut être déterminée grâce aux équations suivantes:

$$x(t) = x_0 + w_x t - 60(w_x - v_{0x})(1 - e^{-t/60})$$

$$y(t) = y_0 + w_y t - 60(w_y - v_{0y})(1 - e^{-t/60})$$

où "t" est en minutes. (x_0, y_0) est la position initiale du mouvement. (v_{0x}, v_{0y}) est la vitesse initiale du mouvement et (w_x, w_y) est le vent donné. Il pourrait être utile de se rappeler que, pour un vecteur de grandeur "z" à un angle "θ" donné, les composantes du vecteur sont données par $(z \cos \theta, z \sin \theta)$. Par exemple, la vitesse initiale d'Ulysse en début de problème est donnée par l'expression $(v_0 \cos 45, v_0 \sin 45)$ comme **il commence toujours par se diriger vers le Nord-Est. Les vents vous seront aussi donnés sous forme cardinale et il en reviendra à vous d'extrapoler leurs composantes.** Vous aurez aussi besoin des équations pour obtenir la vitesse du bateau au temps "t" advenant un changement de vent en pleine mer: **(dans ce cas, votre vitesse finale à la fin d'une période de vent constant devient votre vitesse initiale pour la prochaine période de vent constant)**

$$v_x(t) = w_x - (w_x - v_{0x})e^{-t/60}$$

$$v_y(t) = w_y - (w_y - v_{0y})e^{-t/60}$$

Finalement, c'est bien beau mettre des trajectoires bout-à-bout pour calculer le temps requis pour se rendre à Ithaca, mais il faudra aussi prendre en compte d'autres îles de l'archipel qui peuvent impacter la trajectoire. Si le bateau entre en collision avec une île, **il s'échoue et ne peut repartir de l'île avant qu'un vent ne lui soit favorable**, c'est-à-dire que si le bateau est échoué sur la côte nord d'une île, par exemple, il nécessitera un vent avec une composante pointant vers le Nord pour repartir (soit "NO", "N")

ou "NE"). Le temps perdu sur l'île à attendre un tel vent devra être comptabilisé dans la durée du voyage. Les îles sont des rectangles décrits par 4 points dans le plan cartésien. Le bateau repart à l'arrêt d'une île (sa vitesse initiale est nulle). Les îles, qui sont constantes d'un exemple à l'autre, peuvent être copiées ici:

```
islands=[[ [10,32],[30,32],[30,50],[10,50]],
           [[63,15],[94,15],[94,31],[63,31]],
           [[3,76],[24,76],[24,95],[3,95]],
           [[35,2],[57,2],[57,9],[35,9]],
           [[47,61],[79,61],[79,82],[47,82]]]
```

Le voyage d'Ulysse s'arrêtera à la minute où le bateau se situe sur l'île d'Ithaca:

```
ithaca=[[90,90],[100,90],[100,100],[90,100]]
```

Vous devrez vérifier s'il y a collision avec Ithaca ou une autre île à chaque MINUTE du voyage. Les formules sont données en minutes et les exemples ont été calculés avec la minute comme unité de base: si vous utilisez autre chose que la minute, vous obtiendrez un résultat différent pour la durée du voyage!

Spécifications d'entrée:

En entrée vous recevrez:

- ***v0***: un int qui correspond à la vitesse initiale en km/h du bateau d'Ulysse vers le Nord-Est
- ***winds***: une liste en deux dimensions qui correspond à la séquence de vents affectant le bateau d'Ulysse en ordre chronologique. Chaque vent ***wind*** est une liste de la forme [***force***, ***direction***, ***duration***]. ***force*** est un entier correspondant la vitesse du vent en km/h. ***direction*** est une string de 1 ou 2 caractère(s) symbolisant un point cardinal ("N", "E", "S", "O") ou un des points intercardinaux ("NE", "SE", "SO", "NO") vers lequel souffle le vent. Finalement, ***duration*** est un entier qui indique la durée en heures du vent en question.

Spécifications de sortie:

En sortie vous devrez fournir:

- Un float arrondi au centième près correspondant au nombre total d'heures écoulées depuis le début du voyage (depuis qu'Ulysse est parti du point (0,0)) au moment où il arrive sur l'île d'Ithaca

Exemple d'entrée:

1. 25, [[16,"E",5],[17,"NE",3],[11,"NO",3],[29,"E",2],[42,"N",5],[12,"SO",7]]
2. 47, [[12,"NE",3],[7,"E",5],[18,"N",9]]
3. 5, [[55,"NE",7],[31,"SE",5],[60,"S",3],[19,"N",4],[37,"S",7]]
4. 29, [[37,"E",1],[15,"NO",6],[15,"NE",2],[23,"O",2],[36,"E",1],[31,"N",1],[46,"E",17]]

Exemple de sortie:

1. 15.05
2. 10.68

3. 19.13

4. 14.95

Explication de la première sortie:

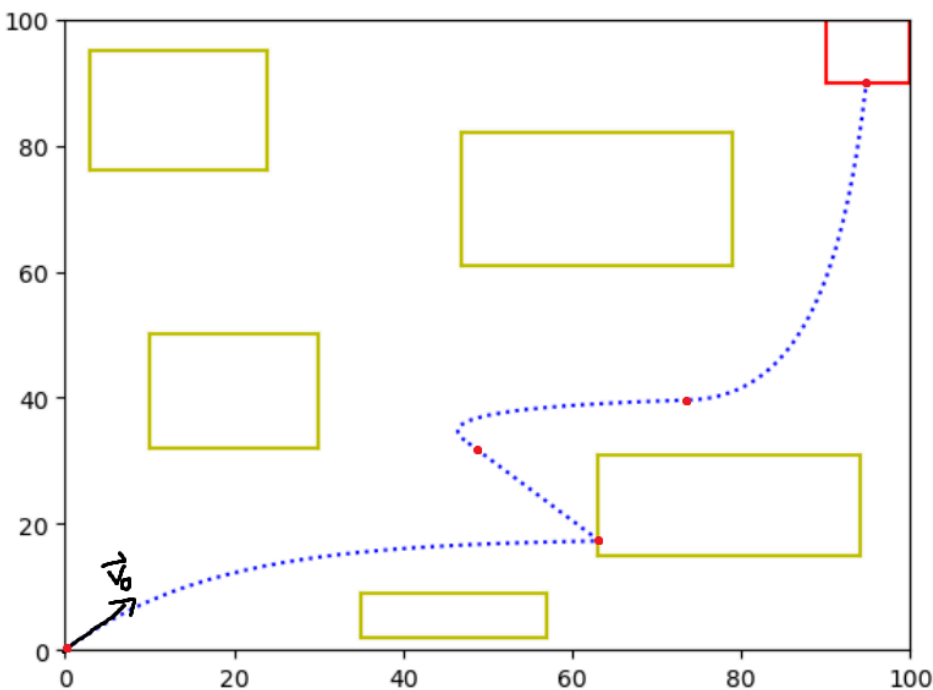
Le premier vent souffle vers l'Est pendant 5 heures, ce qui fait échouer le bateau sur la plage ouest de l'île jaune la plus à droite selon nos calculs.

Après ces 5 heures, le vent change, mais c'est un vent vers le Nord-Est qui souffle! Comme le vent souffle le bateau vers la terre de l'île au lieu de la mer, il ne peut pas faire progresser le bateau et Ulysse est coincé sur cette île pendant 3 autres heures.

Enfin, un vent vers le Nord-Ouest se lève à raison de 11 km/h et permet à Ulysse et ses compagnons de repartir lentement d'où ils s'étaient échoués ainsi que d'avancer un peu sur la mer. Comme on ne rencontre aucune île sous ce vent, nous sommes encore en pleine mer lorsque le vent change et il nous faut donc calculer la vitesse du bateau à la fin des trois heures pour l'utiliser comme vitesse initiale pour le prochain vent.

Ensuite, un vent souffle le bateau vers l'Est et change complètement sa trajectoire. Comme nous sommes encore en mer après les 2 heures de ce vent, nous répétons ce processus.

Finalement, un vent puissant qui se lève vers le Nord permet à Ulysse et à ses compagnons de rentrer à Ithaca après 2 heures et 3 minutes supplémentaires. Comme ils sont rentrés, le dernier vent de la liste n'a pas d'importance. Le temps total de voyage était de $5+3+3+2+2.05=15.05$ heures.



Les rectangles jaunes sont les îles sur la mer. Le rectangle rouge est l'île d'Ithaca, soit la destination requise. Ulysse part du point (0,0) avec une vitesse initiale vers le Nord-Est. Chaque point rouge correspond approximativement à un début et/ou à une fin de période de vent significatif pendant le trajet. Le pointillé bleu est le trajet en question.

Olympique d'hiver (WO)

WO1: Patinage de vitesse (25 points)

Les jeux olympiques d'hiver ont plusieurs sports comme le patinage de vitesse. Il faut donc déterminer qui remportera la course et quel sera leur temps à la fin selon plusieurs statistiques.

Les vitesses sont en km/h.

Spécifications d'entrée:

En entrée vous recevrez:

- ***distance***: un *int* qui représente la distance en mètres à patiner
- ***skaters***: un *array* de *string* avec le nom des patineurs
- ***initial_speed***: un *array* de *float* qui est la vitesse initiale des patineurs
- ***stamina***: un *array* de *int* qui indique le nombre de mètres que le patineur peut aller à sa vitesse initiale
- ***second_speed***: un *array* de *float* qui est la vitesse après épuisement de la stamina des patineurs

Un même indice pour différents *arrays* correspond au même patineur.
i=0 dans chaque *array* concerne les statistiques du premier patineur

Spécifications de sortie:

En sortie vous devrez fournir:

- Une *string* avec le nom du vainqueur et son temps en secondes arrondi au centième près
ex: "*nom, temps.00s*"

Exemple d'entrée:

```
distance = 500
skaters = ["Serina", "Beatrice", "Jutta"]
initial_speed = [48.26, 53.35, 48.45]
stamina = [750, 400, 600]
second_speed = [47.23, 34.16, 38.45]
```

```
distance = 1000
skaters = ["Courtney", "Li", "Gilli", "Arianna"]
initial_speed = [44.10, 43.90, 43.36, 43.70]
stamina = [750, 750, 625, 625]
second_speed = [19.79, 19.36, 27.10, 26.86]
```

```
distance = 2400
skaters = ["United States", "Japan", "Canada", "Pays-Bas"]
initial_speed = [48.19, 48.41, 54.61, 49.03]
stamina = [1600, 1200, 2000, 1800]
second_speed = [46.10, 48.40, 48.93, 29.32]
```

Exemple de sortie:

"Jutta, 37.15s"

"Gilli, 101.71s"

"Canada, 161.27s"

Explication de la première sortie:

Dans la première entrée il y a 3 patineuses, Serina a fait un temps de 37.28s, Béatrice a fait un temps de 26.99 à son plus rapide et un temps de 10.54s à son plus lent ce qui fait un temps de 27.53s et Jutta a fait un temps de 37.15s qui est calculé en utilisant la formule

$\text{Temps} = \text{distance} / \text{vitesse} \times 3.6$

Donc le calcul à effectuer est $500 / 48.45$, qui donne environ 10.3199, soit un temps de 37.15s après le $\times 3.6$.

WO2: Bobsleigh (45 points)

Le bobsleigh est un sport des plus rapide lors des jeux olympiques avec des vitesses dépassant les 130, 140 et même parfois 150 km/h!! À cette vitesse-là, chaque décision doit être étudiée à l'avance et il faut connaître la piste par cœur.

Vous devrez faire un programme pour aider les athlètes en regardant la piste, puis en donnant des instructions pour chaque segment de piste. Si on ne change pas de direction, on affiche R, si on tourne vers la gauche, on affiche la lettre G et, finalement, si on doit tourner vers la droite, on affiche la lettre D.

Spécifications d'entrée:

En entrée vous recevrez:

- *path*: un *array* de *strings* qui représentent la piste

Spécifications de sortie:

En sortie vous devrez fournir:

- une *string* qui représente les direction à prendre, R étant rien, G étant gauche et D étant droite

Exemple d'entrée:



Exemple de sortie:

RRRRGRRRRGRDRDRGRGRDRRRDRDRGRDRGRGRDRDRRRRRDRRGRRRGRGRDRRR

Explication de la première sortie:

La descente commence au D et finit au F. Quand le bobsleigh va tout droit, il ne faut rien faire, mais il ne faut pas oublier de tourner à droite ou à gauche dans le but de ne pas crasher le bob.

WO3: Saut à ski (15 points)

Lors d'une compétition de saut à ski aux Jeux olympiques d'hiver, chaque athlète effectue un saut qui est évalué selon deux critères :

- la distance du saut
- les notes données par les juges

Le score final d'un saut est calculé selon les règles suivantes :

1. Chaque mètre de distance rapporte **2 points**
2. Cinq juges donnent chacun une note entière entre **0 et 20**
3. La **meilleure** et la **pire** note sont retirées
4. Les **trois notes restantes** sont additionnées
5. Le score final est la somme :
 - des points de distance
 - des points des juges retenus

Spécifications d'entrée:

En entrée vous ne recevrez:

- **distance:** un *int* de la distance en mètre parcourue par l'athlète
- **scores:** un *array* de *int* entre 0 et 20 des notes des 5 juges

Spécifications de sortie:

En sortie vous devrez fournir:

- un *int* du score total du saut

Exemple d'entrée:

```
distance = 120
```

```
scores = [18, 19, 17, 20, 16]
```

```
distance = 95
```

```
scores = [15, 15, 16, 14, 15]
```

```
distance = 130
```

```
scores = [20, 20, 19, 18, 17]
```

Exemple de sortie:

294

235

317

Explication de la première sortie:

Points de distance : $120 \times 2 = 240$

Notes triées : [16, 17, 18, 19, 20]

On retire 16 et 20

Somme des notes retenues : $17 + 18 + 19 = 54$

Score final : $240 + 54 = 294$

WO4: Il faut être le meilleur pour une médaille? Right? (30 points)

Un tournoi olympique regroupe 16 équipes. Chaque équipe possède une cote de force entière entre 1 et 100.

Le tournoi est organisé sous forme d'**arbre de tournoi à élimination unique** :

- 16 équipes → 8 gagnants
- 8 équipes → 4 gagnants
- 4 équipes → 2 gagnants
- Finale

Les demi-finalistes (top 4) se disputent les médailles :

- les deux finalistes obtiennent **or et argent**
- les deux autres demi-finalistes compétitionnent pour la **médaille de bronze**

Donc, une équipe obtient une médaille si elle fait partie du **top 3** (finaliste ou gagne le match de bronze).

Règles:

- Si deux équipes s'affrontent, **celle avec la cote la plus élevée gagne toujours**.
- Toutes les cotes sont différentes.
- Vous pouvez organiser les brackets **de manière complètement libre** au début du tournoi.
- Vous choisissez le placement initial des 16 équipes.
- Vous devrez déterminer s'il existe un placement pour lequel l'équipe choisie peut gagner une médaille

Vous retournez 1 si l'équipe choisie peut gagner l'or, 2 si elle peut potentiellement gagner l'argent, 3 si elle peut potentiellement gagner le bronze et 0 si elle ne peut pas gagner de médaille peu importe l'organisation des brackets.

Spécifications d'entrée :

En entrée vous recevrez:

- **ratings** : un *array* de *int* qui sont la cote de force de chaque équipe
- **team**: un *int* de la position de l'équipe dans le tableau

Spécifications de sortie :

En sortie vous devrez fournir:

- un *int* de la meilleure position possible d'atteindre avec un positionnement de tournoi optimal et, si ce n'est pas une des 3 premières positions, la valeur 0.

Exemple d'entrée :

```
[100, 95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25]  
8
```

```
[95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25, 20]  
5
```

```
[100, 95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25]  
0
```

Exemple de sortie :

```
2  
2  
1
```

Explication de la première sortie :

Avec le premier exemple, nous souhaitons organiser pour amener l'équipe avec un score de 60 à une médaille. 8 équipes sont meilleures et 7 sont moins fortes que notre équipe choisie. Voici une répartition pour atteindre la médaille d'argent:

100 vs 95	60 vs 55
90 vs 85	50 vs 45
80 vs 75	25 vs 35
70 vs 65	40 vs 20

100 vs 90	60 vs 50
80 vs 70	35 vs 40

100 vs 80	60 vs 40
-----------	----------

Grande Finale: 100 vs 60

Match 3e place: 80 vs 40

Médaille d'or: 100

Médaille d'argent: 60

Médaille de bronze: 80

On peut voir ici que même en étant pas du tout l'équipe la plus forte, il existe moyen d'espérer pour une médaille Olympique!! Cette fois-ci notre équipe avec un score de 60 obtient la médaille d'Argent!

WO5: Parfaite ligne d'attaquants (55 points)

Une équipe de hockey compte souvent dans un match 12 attaquants, 6 défenseurs et 2 gardiens: 4 lignes de 3 attaquants, 3 paires de défenseurs avec 1 gardien dans les buts et un gardien en cas de besoin.

Bien qu'une grande partie de la stratégie se joue sur la glace, il est important de bien choisir la position de chaque joueur pour former le meilleur alignement possible.

Dans ce problème vous devrez former la meilleure équipe en répartissant les 12 attaquants sur les 4 lignes d'attaque. Chaque attaquant aura 2 statistiques à prendre en compte, leur endurance ainsi que leur talent naturel. Leur score d'efficacité sera un produit de leur talent naturel et leur endurance qui sera variable selon la ligne sur laquelle ils sont. Vous devrez placer les 4 ailiers gauches sur les 4 lignes d'attaque, les 4 centres et les 4 ailiers droit pareillement. Votre objectif est de former la meilleure formation d'attaque pour maximiser l'efficacité.

Les joueurs jouant sur la première ligne sont ceux qui ont un plus grand impact sur leur match par leur grande présence. Leur score d'efficacité se calcule ainsi:

$$efficacity = talent * 1.2 + endurance * 1.2$$

Les joueurs jouant sur la seconde ligne sont souvent très talentueux, mais jouent un peu moins de temps de match. Leur score d'efficacité se calcule ainsi:

$$efficacity = talent * 1.2 + endurance$$

Les joueurs jouant sur la troisième ligne sont souvent un peu moins fort que les deux premières lignes. Leur score d'efficacité se calcule ainsi:

$$efficacity = talent + endurance$$

Les joueurs jouant sur la quatrième et dernière ligne sont souvent un peu moins talentueux et leur rôle est souvent d'être très énergique pour donner du moral au reste de l'équipe. Leur score d'efficacité se calcule ainsi:

$$efficacity = talent * 0.6 + endurance * 1.2$$

Vous devrez fournir en ordre les 4 lignes avec le numéro de chandail de l'ailier gauche, puis du centre et de l'ailier droit pour chaque ligne d'attaque dans l'objectif de maximiser le score d'efficacité de l'équipe.

Spécifications d'entrée:

En entrée vous ne recevrez:

- ***left_wing***: un *array* des quatres joueurs d'aile gauche avec leur numéro suivi de leur score de talent puis de leur score d'endurance
- ***center***: un *array* des quatres joueurs avec leur numéro suivi de leur score de talent puis de leur score d'endurance
- ***right_wing***: un *array* des quatres joueurs avec leur numéro suivi de leur score de talent puis de leur score d'endurance

Spécifications de sortie:

En sortie vous devrez fournir:

- un *array* de *array* de int qui sont les numéros des chandails (gauche, centre droite) pour les 4 lignes.

Exemple d'entrée:

```
l_w = [(26, 8, 8), (98, 10, 9), (56, 7, 6), (44, 6, 7)]
```

```
c = [(23, 7, 7), (66, 6, 7), (8, 9, 10), (12, 8, 8)]
```

```
r_w = [(41, 4, 8), (74, 9, 9), (33, 6, 6), (55, 9, 10)]
```

```
l_w = [(11, 10, 5), (22, 5, 10), (33, 8, 8), (44, 3, 3)]
```

```
c = [(55, 9, 6), (66, 6, 9), (77, 7, 7), (88, 4, 4)]
```

```
r_w = [(91, 10, 10), (92, 2, 8), (93, 8, 2), (94, 5, 5)]
```

```
l_w = [(1, 7, 8), (2, 8, 7), (3, 6, 6), (4, 5, 5)]
```

```
c = [(5, 9, 8), (6, 8, 9), (7, 7, 7), (8, 6, 6)]
```

```
r_w = [(9, 10, 7), (10, 7, 10), (11, 8, 8), (12, 5, 6)]
```

Exemple de sortie:

```
[[98, 8, 55], [26, 12, 74], [56, 23, 33], [44, 66, 41]]
```

```
[[33, 66, 91], [11, 55, 93], [44, 77, 94], [22, 88, 92]]
```

```
[[1, 6, 10], [2, 5, 9], [3, 7, 11], [4, 8, 12]]
```


Explication de la première sortie:

Oui c'est la réponse donnée par chat gpt (non vous n'avez PAS le droit de l'utiliser durant la compétition désolé) mais bon, l'explication est très bien structurée...

Étape 1 : Comprendre le calcul de l'efficacité par ligne

Chaque ligne a une formule différente pour l'efficacité :

Ligne	Formule
1ère	$\text{talent} * 1.2 + \text{endurance} * 1.2$
2ème	$\text{talent} * 1.2 + \text{endurance}$
3ème	$\text{talent} + \text{endurance}$
4ème	$\text{talent} * 0.6 + \text{endurance} * 1.2$

Observation :

- La **1ère ligne** valorise le talent et l'endurance le plus (multipliés par 1.2).
- La **2ème ligne** valorise davantage le talent que l'endurance.
- La **3ème ligne** est équilibrée.
- La **4ème ligne** favorise davantage l'endurance que le talent ($0.6 \times \text{talent}$, $1.2 \times \text{endurance}$).

Donc la stratégie est :

- Mettre la **meilleure combinaison talent+endurance** sur la **1ère ligne**.
- Mettre des joueurs légèrement moins forts sur les lignes 2 et 3.
- Mettre des joueurs avec un talent plus faible mais une bonne endurance sur la 4ème ligne.

Étape 2 : Calculer le score "effectif" pour chaque joueur

Pour la clarté, calculons les **scores de la 1ère ligne** pour tous les joueurs :

Ailiers gauches (l_w) pour la formule de la 1ère ligne : $\text{talent}1.2 + \text{endurance}1.2$

- $26 \rightarrow 81.2 + 81.2 = 9.6 + 9.6 = 19.2$
- $98 \rightarrow 101.2 + 91.2 = 12 + 10.8 = 22.8$ ✓
- $56 \rightarrow 71.2 + 61.2 = 8.4 + 7.2 = 15.6$
- $44 \rightarrow 61.2 + 71.2 = 7.2 + 8.4 = 15.6$

Centres (c) pour la 1ère ligne :

- $23 \rightarrow 71.2 + 71.2 = 8.4 + 8.4 = 16.8$
- $66 \rightarrow 61.2 + 71.2 = 7.2 + 8.4 = 15.6$
- $8 \rightarrow 91.2 + 101.2 = 10.8 + 12 = 22.8$ ✓
- $12 \rightarrow 81.2 + 81.2 = 9.6 + 9.6 = 19.2$

Ailiers droits (r_w) pour la 1ère ligne :

- $41 \rightarrow 41.2 + 81.2 = 4.8 + 9.6 = 14.4$
- $74 \rightarrow 91.2 + 91.2 = 10.8 + 10.8 = 21.6$
- $33 \rightarrow 61.2 + 61.2 = 7.2 + 7.2 = 14.4$
- $55 \rightarrow 91.2 + 101.2 = 10.8 + 12 = 22.8$ ✓

Observation :

- **Meilleur ailier gauche :** 98
- **Meilleur centre :** 8
- **Meilleur ailier droit :** 55

✓ Cela correspond à la **première ligne de la sortie** : [98, 8, 55]

Étape 3 : Assigner la 2ème ligne

Formule 2ème ligne : talent*1.2 + endurance

On enlève les joueurs de la 1ère ligne :

- Ailiers gauches restants : 26, 56, 44
- Centres restants : 23, 66, 12
- Ailiers droits restants : 41, 74, 33

Scores approximatifs :

Ailiers gauches :

- $26 \rightarrow 8*1.2 + 8 = 9.6 + 8 = 17.6$ ✓
- $56 \rightarrow 7*1.2 + 6 = 8.4 + 6 = 14.4$
- $44 \rightarrow 6*1.2 + 7 = 7.2 + 7 = 14.2$

Centres :

- $23 \rightarrow 7*1.2 + 7 = 8.4 + 7 = 15.4$
- $66 \rightarrow 6*1.2 + 7 = 7.2 + 7 = 14.2$
- $12 \rightarrow 8*1.2 + 8 = 9.6 + 8 = 17.6$ ✓

Ailiers droits :

- $41 \rightarrow 4 \cdot 1.2 + 8 = 4.8 + 8 = 12.8$
- $74 \rightarrow 9 \cdot 1.2 + 9 = 10.8 + 9 = 19.8$ ✓
- $33 \rightarrow 6 \cdot 1.2 + 6 = 7.2 + 6 = 13.2$

Meilleure combinaison pour la 2ème ligne : [26, 12, 74] ✓ Correspond à la sortie.

Étape 4 : Assigner la 3ème ligne

Formule 3ème ligne : talent + endurance

Joueurs restants :

- Ailiers gauches : 56, 44
- Centres : 23, 66
- Ailiers droits : 41, 33

Calcul des sommes :

- Gauches : $56 \rightarrow 7+6=13$, $44 \rightarrow 6+7=13$
- Centres : $23 \rightarrow 7+7=14$, $66 \rightarrow 6+7=13$
- Droites : $41 \rightarrow 4+8=12$, $33 \rightarrow 6+6=12$

Meilleure combinaison totale :

- Gauche : 56
- Centre : 23
- Droite : 33

✓ Correspond à la sortie : [56, 23, 33]

Étape 5 : Assigner la 4ème ligne

Joueurs restants : [44, 66, 41]

- Formule 4ème ligne : talent 0.6 + endurance 1.2
- Gauche : $44 \rightarrow 6 \cdot 0.6 + 7 \cdot 1.2 = 3.6 + 8.4 = 12$
- Centre : $66 \rightarrow 6 \cdot 0.6 + 7 \cdot 1.2 = 3.6 + 8.4 = 12$
- Droite : $41 \rightarrow 4 \cdot 0.6 + 8 \cdot 1.2 = 2.4 + 9.6 = 12$

✓ Correspond à la sortie : [44, 66, 41]

✓ Étape 6 : Résumé

- **Ligne 1 (titulaire) :** [98, 8, 55] → meilleur talent + endurance
- **Ligne 2 :** [26, 12, 74] → combinaison suivante la plus forte
- **Ligne 3 :** [56, 23, 33] → ligne moyenne équilibrée
- **Ligne 4 (énergie/morale) :** [44, 66, 41] → joueurs plus faibles, endurance privilégiée